Implementation Document

for

IITKart

Version 1.02

Prepared by

Group 5: **Group Name: Thunderbolts**

Hardik Tiwari	230437	hardikt23@iitk.ac.in
Pranjali Deshpande	240336	deshpandep24@iitk.ac.in
Mahi Mittal	240608	mahii24@iitk.ac.in
Mayukh Chowdhury	240336	cmayukh@iitk.ac.in
Palak Bandhu	240719	palakb24@iitk.ac.in
Praveen	240790	praveenas24@iitk.ac.in
Shreshthraj Bhidodiya	240996	sbhidodiya24@iitk.ac.in
Sneha Kumari	241024	snehak24@iitk.ac.in
Virendra Kala	241172	virendrak24@iitk.ac.in
Yash Dabi	241195	yashd24@iitk.ac.in

Course: CS253

Mentor TA: *George T L*

Date: 27/03/2026

CONTENTS II
REVISIONS II

1 IMPLEMENTATION DETAILS 1

2 CODEBASE 2

3 COMPLETENESS 3

APPENDIX A - GROUP LOG 4

Revisions

Version	Primary Author(s)	Description of Version	Date Completed
1.00	Thunderbolts	**Initial Prototype (IITKart):** First iteration focused on basic UI structure and user flow. This version was ultimately rejected due to architectural constraints and scalability issues with the backend integration.	24/03/2026
1.01	Thunderbolts	**Alpha Build (Demo Repo):** Migration to the `Demo` repository. Implemented the core MERN-like stack (React, Express, Prisma). Established the relational database schema and basic Vendor-Product CRUD operations.	26/03/2026
1.02	Thunderbolts	**Stable Release (Final IITKart):** Resolved critical bugs related to database schema conflicts and Vite build errors. Optimized JWT authentication flow and finalized the Vendor Dashboard. Synchronized via a full database reset on March 27.	27/03/2026

1 Implementation Details

1. Programming Language: TypeScript

- **Role:** Used across both the Frontend and Backend (Node.js).
- **Justification:** Unlike standard JavaScript, **TypeScript** provides static type-checking. This was critical for our project to catch "undefined" errors during development rather than at runtime. It also allowed us to share "Interfaces" between the frontend and backend, ensuring that a `Product` object sent by the vendor always matched what the database expected.

2. Frontend Framework: React (with Vite)

- **Role:** Building the User and Vendor interfaces.
- **Justification: React's** component-based architecture allowed us to build the "Vendor Dashboard" and "Product Upload" forms as reusable modules. We chose **Vite** as the build system over *Create React App* because Vite offers significantly faster Hot Module Replacement (HMR), which was essential for our rapid iteration leading up to the deadline.

3. Backend Framework: Express.js

- **Role:** Handling API requests, authentication middleware, and routing.
- **Justification: Express** is a minimalist and flexible framework. It allowed us to quickly implement custom middleware for JWT verification and vendor-specific route protection without the overhead of heavier frameworks like NestJS or Django.

4. Database System: PostgreSQL

- **Role:** Relational storage for Users, Vendors, and Products.
- **Justification:** Since our software requires strict relationships (e.g., a Product **must** belong to a specific Vendor), a **Relational Database (RDBMS)** like **PostgreSQL** was superior to NoSQL alternatives like MongoDB. PostgreSQL ensures **ACID compliance**, meaning transactions like order placements are processed reliably.

5. ORM: Prisma

- **Role:** Database schema management and query building.
- **Justification: Prisma** acts as a bridge between our TypeScript code and the SQL database. We chose it over *Sequelize* or raw SQL because it auto-generates a type-safe client. When we defined the `Product` model (e.g., for the "Iced Coffee"

entry), Prisma ensured our code strictly followed that schema, preventing database corruption.

6. State Management & Libraries

- **React Context API:** Used for global state management (handling `currentUser` and `auth` status) instead of Redux. Since our state isn't overly complex yet, Context API provided a built-in, lightweight solution without adding external dependencies.
- **JSON Web Tokens (JWT):** Used for secure, stateless authentication. This is more scalable than session-based auth because the server doesn't need to store session IDs in memory.
- **Axios:** Used for making asynchronous API calls from the frontend to the backend due to its better error-handling capabilities compared to the native `fetch` API.

2 Codebase

GitHub Repository: https://github.com/palakbandhu/IITKart

Code Structure

The codebase of IITKart is divided into two parts : Backend and frontend

Backend structure

```
backend/
├── src/
│   ├── config/            # Configuration files
│   │   ├── env.ts         # Environment variable validation (Zod/Dotenv)
│   │   ├── cors.ts        # CORS security options
│   │   └── db.ts          # Database connection initialization
│   │
│   ├── controllers/       # Request Handlers (Entry point for API calls)
│   │   ├── authController.ts # Login, Register, OTP, Password Reset
│   │   ├── userController.ts # Profile, Wallet, Address management
│   │   ├── vendorController.ts # Menu management, Shop status, Analytics
│   │   ├── riderController.ts  # Delivery requests, Status toggling
│   │   ├── adminController.ts  # Platform moderation, User management
│   │   ├── orderController.ts  # Order placement, Tracking, Status updates
│   │   └── paymentController.ts # Payment gateway webhooks
│   │
│   ├── middlewares/       # Express Middlewares
│   │   ├── authMiddleware.ts # JWT verification & RBAC (Role-Based Access Control)
│   │   ├── errorMiddleware.ts # Global error handling wrapper
│   │   ├── validateRequest.ts # Request body schema validation (Zod)
│   │   └── uploadMiddleware.ts # File upload handling (Multer)
│   │
│   ├── routes/            # API Route Definitions
│   │   ├── authRoutes.ts     # /api/auth
│   │   ├── userRoutes.ts     # /api/users
│   │   ├── vendorRoutes.ts   # /api/vendors
│   │   ├── riderRoutes.ts    # /api/riders
│   │   ├── adminRoutes.ts    # /api/admin
│   │   └── orderRoutes.ts    # /api/orders
│   │
│   ├── services/          # Business Logic Layer
│   │   ├── authService.ts    # Hashing, Token generation logic
│   │   ├── orderService.ts   # Complex order processing logic
│   │   ├── paymentService.ts # Interactions with Payment Gateway APIs
│   │   └── notificationService.ts # Email/SMS/Push notification triggers
│   │
```

```
│   ├── utils/              # Shared Utilities
│   │   ├── AppError.ts       # Custom error class
│   │   ├── logger.ts        # Logging configuration (Winston/Morgan)
│   │   └── helpers.ts       # Common helper functions
│   │
│   ├── app.ts              # Express app configuration (Middlewares, Routes)
│   └── server.ts           # Server entry point (Port listening)
│
├── prisma/                 # Database ORM
│   ├── schema.prisma        # PostgreSQL Schema definition
│   ├── migrations/          # SQL Migration history
│   └── seed.ts             # Script to populate DB with dummy data
│
├── tests/                  # Unit and Integration Tests
├── .env                    # Environment secrets (DB_URL, JWT_SECRET)
├── package.json            # Backend dependencies
└── tsconfig.json           # TypeScript config
```

Frontend structure

```
frontend/
├── public/                 # Static Assets (Favicon, manifest.json)
│
├── src/
│   ├── assets/             # Source Assets (Images, Icons, Fonts)
│   │
│   ├── components/          # UI Components
│   │   ├── ui/             # Reusable atomic elements (Buttons, Inputs, Modals)
│   │   ├── layout/          # Structural components (Navbar, Sidebar, Footer)
│   │   ├── common/          # Generic components (Loaders, ErrorBoundaries)
│   │   └── features/        # Feature-specific components
│   │       ├── cart/        # Cart drawer, Cart items
│   │       ├── map/         # Map integration components
│   │       └── products/     # Product cards, Filters
│   │
│   ├── context/            # Global State Management (React Context)
│   │   ├── AuthContext.tsx   # User session state
│   │   ├── CartContext.tsx   # Shopping cart state
│   │   └── SocketContext.tsx # WebSocket connection for real-time updates
│   │
│   ├── hooks/              # Custom React Hooks
│   │   ├── useAuth.ts
│   │   ├── useFetch.ts
│   │   └── useDebounce.ts
│   │
│   ├── layouts/            # Page Layout Wrappers
```

```
│   │   ├── MainLayout.tsx    # Standard user layout
│   │   ├── DashboardLayout.tsx # Layout for Vendors/Admin
│   │   └── AuthLayout.tsx    # Layout for Login/Register pages
│   │
│   ├── pages/              # Application Views (Routes)
│   │   ├── auth/           # Login, Signup, Forgot Password
│   │   ├── customer/       # Home, Restaurant Menu, Checkout, Profile
│   │   ├── vendor/         # Dashboard, Menu Editor, Order History
│   │   ├── rider/          # Active Delivery, Earnings, History
│   │   └── admin/          # Analytics, User Management
│   │
│   ├── services/           # API Client (Axios/Fetch)
│   │   ├── api.ts          # Axios instance with interceptors
│   │   └── endpoints/      # API call functions grouped by domain
│   │
│   ├── types/              # TypeScript Interfaces/Types
│   │   ├── models.ts       # User, Order, Product interfaces
│   │   └── apiResponse.ts    # Standard API response types
│   │
│   ├── utils/              # Frontend Utilities
│   │   ├── formatCurrency.ts
│   │   ├── formatDate.ts
│   │   └── validators.ts
│   │
│   ├── App.tsx             # Main Component & Route Setup
│   ├── main.tsx            # Entry Point
│   └── index.css           # Global Styles & Tailwind Directives
│
├── .env                    # Frontend Environment Variables (API URL)
├── package.json            # Frontend dependencies
├── tailwind.config.js      # Tailwind configuration
├── tsconfig.json           # TypeScript config
└── vite.config.ts          # Vite bundler config
```

Backend

The backend is built with **Node.js**, **Express**, and **TypeScript**, following a standard MVC architecture. The entry point is Backend1/src/server.ts, which initializes the Express app, loads middleware, and registers all API routes.
Database interactions are handled entirely through **Prisma ORM**, with **PostgreSQL** as the underlying database. The schema is defined in Backend1/prisma/schema.prisma and covers all core models including Users, VendorProfiles, RiderProfiles, Products, Orders, OrderItems, Cart, and CartItems.
The routes directory contains seven route files, each corresponding to a distinct domain of the application: authentication, users, vendors, riders, orders, payments, and admin. Each

route file connects to its respective controller, which contains the business logic for that domain.
Authentication is handled via JWT tokens, with separate secrets and expiry configurations for access tokens and refresh tokens. Role-based access control is enforced through a custom authMiddleware that protects routes and restricts access based on user roles (CUSTOMER, VENDOR, RIDER, ADMIN).
The backend also integrates with Razorpay for payment processing and uses SMTP configuration for email delivery (used for OTP and notifications).
Key environment variables required to run the backend:

Variable	Purpose
DATABASE_URL	PostgreSQL connection string for Prisma ORM
JWT_SECRET	Secret key for signing authentication access tokens
JWT_EXPIRES_IN	Duration before the access token becomes invalid
REFRESH_TOKEN_SECRET	Secondary secret used to issue new access tokens
REFRESH_TOKEN_EXPIRES_IN	Duration before a user is forced to log in again

PORT	The local or production port the Express server listens on
NODE_ENV	Defines environment context (e.g., development, production)
FRONTEND_URL	Configures CORS to allow requests from the React Vite app
RAZORPAY_KEY_ID	Public API key for the Razorpay payment gateway
RAZORPAY_KEY_SECRET	Private secret for verifying Razorpay transactions
SMTP_HOST / SMTP_PORT	Configuration settings for the outgoing email server
SMTP_USER / SMTP_PASS	Credentials for the system's automated email account
FROM_EMAIL	The default sender address for system notifications

To run the backend development server:
cd Backend1
npm run dev

To apply database migrations:
npm run db:migrate

Frontend

The frontend is built with React and TypeScript, bundled using Vite (v6.3.5). The entry point is frontend/src/main.tsx.
The UI is organized around three role-specific interface components, each representing the experience for a different type of user on the platform. UserInterfaceNew.tsx serves the customer-facing experience, VendorInterfaceNew.tsx handles the vendor dashboard and shop management, and CourierInterface.tsx covers the rider's delivery workflow. Styling is handled through a combination of Tailwind CSS (v4.1.12) and Material UI (v7.3.5). Navigation between views is managed using React Router DOM (v7.12.0), and icons are sourced from the Lucide React library (v0.487.0).
The frontend communicates with the backend via a single base URL configured through the VITE_API_URL environment variable.

To run the frontend development server:
cd frontend
npm run dev

Dependency Stack Summary

Layer	Technology
Runtime	Node.js
Framework	Express + TypeScript

ORM	Prisma
Database	PostgreSQL
Bundler	Vite
UI Components	Material UI
Styling	Tailwind CSS
Routing	React Router DOM
Icons	Lucide React
Payments	Razorpay

3 Completeness

3.1 Completed Features from SRS

The current implementation successfully covers the core functional requirements for user onboarding, vendor operations, and product visibility as specified in the SRS.

Feature ID	Feature Name	Description	Status
FR-1	**User Authentication**	Registration and Login for both Customers and Vendors using JWT-based security.	**Completed**
FR-2	**Vendor Onboarding**	Ability for users to register as Vendors and access the dedicated Vendor Dashboard.	**Completed**
FR-3	**Product Management**	Vendors can add, update, and delete products (e.g., "Iced Coffee") with price and description.	**Completed**
FR-4	**Product Catalog**	Implementation of a dynamic frontend that fetches and displays products from the PostgreSQL database.	**Completed**
NFR-1	**Database Integrity**	Relational mapping via Prisma to ensure every product is linked to a valid Vendor ID.	**Completed**

Verification Note: All completed features have undergone unit testing for backend controllers and manual integration testing using the React frontend to ensure data persistence in the database.

3.2 Future Development Plan

While the current version provides a stable foundation for vendor-product interactions, the following high-priority features are scheduled for future releases:

1. **Multi-Vendor Checkout:**
   Currently, the system handles single-vendor transactions. Future versions will allow customers to add items from multiple different vendors into a single cart and place a unified order, with the backend automatically splitting the orders for respective vendors.
2. **Integrated UPI Payment Gateway:**
   To move beyond the current simulation, we plan to integrate a production-ready UPI interface (using Razorpay or PhonePe API). This will allow users to scan a dynamic QR code or use a VPA handle to complete payments securely within the app.

   The current version includes options of payment by both UPI and CashonDelivery, but UPI payment gateway is yet to be integrated properly.

3. **Kart Coins (Loyalty Program):**
   A gamified reward system where users earn "Kart Coins" for every purchase. These coins will be stored in a digital wallet and can be redeemed for discounts on future orders, incentivizing user retention.
   The current version includes addition of Kart coins after each purchase to Customer's wallet , but does not involve the feature of using Kart coins to avail discounts on orders.
4. **Campus Service Integration (Printing & Laundry):** To fulfill the specific campus utility requirements outlined in the SRS, a dedicated module will be added for high-demand services:

   Photocopy & Digital Printing: Users can upload documents (PDFs) directly via the app, specify preferences such as B&W/Color or double-sided printing, and complete the payment online. The vendor (e.g., a Hall xerox shop) receives the file and a unique "Job ID," enabling the user to pick up their copies without waiting in line.

   Laundromat Services: A specialized integration for the **IITK Laundromat building** will allow students to schedule laundry slots, select service types (Wash/Dry/Steam Press), and track the real-time status of their clothes. By digitizing the queue management for the central Laundromat, the app will reduce physical congestion and provide users with automated notifications when their laundry is ready for collection.

Appendix A - Group Log

Date	Timings	Duration	Minutes
6 Feb,2026	7:00 PM - 12:00 AM	5 hours	Formulated a comprehensive project plan and development timeline using a structured Gantt chart approach. Executed a final, rigorous formatting review of the Design Document to ensure strict alignment with standard software engineering practices.
24 Feb,2026	12:30 AM - 3:30 AM	3 hours	Finalized the robust backend technology stack (Node.js, Express, PostgreSQL, Prisma). Conducted a team-wide technical sync to establish advanced GitHub workflows, including mandatory pull-request reviews and issue tracking for the upcoming development sprints.
3 March,2026	8:00 PM - 11:00 PM	3 hours	Formally divided the team into pairs for the implementation phase, assigning complete ownership of specific API endpoints (Authentication, Orders, Products). Decided to hold mandatory virtual syncs to track progress and collaboratively resolve blockers.
15 March,2026	2:30 PM - 7:30 PM	5 hours	Executed a major backend refactoring sprint. Implemented industrial-grade Zod schema validation, centralized the Prisma singleton architecture to optimize database connections, and integrated Winston logging for professional-grade error tracking.
18 March,2026	5:00 PM - 9:00 PM	4 hours	Distributed frontend development responsibilities across the defined user roles. Held a collaborative design session to outline the logic for a custom semantic Search API, prioritizing product relevance scoring and dynamic filtering.
25 March,2026	12:30 AM - 4:30 AM	4 hours	Initiated the critical system integration phase. Pairs worked synchronously to

			connect the React frontend (Vite/Tailwind) with the secured backend routes. Successfully resolved complex Prisma database migrations and integrated the secure email OTP authentication flow.
26 March,2026	9:30 PM - 1:30 AM	4 hours	Conducted an intensive, cross-functional beta-testing phase. Systematically identified and patched runtime bugs related to the Super Admin dashboard, Razorpay payment gateway verification, active order state reloading, and dynamic session management.
27 March,2026	7:00 PM - 12:00 PM	5 hours	Executed the final code polish and deployment protocols. Conducted end-to-end user flow testing across all three interfaces to ensure a seamless, production-ready experience. Compiled the final codebase and documented the complete system architecture for this Implementation Document.